Java Spring Backend Optimization Test

Background

You are working on an existing tutorial management system built with Java Spring Boot. The system allows users to retrieve information about tutorials along with their rankings. The current implementation has performance issues, especially when retrieving tutorials with their average rankings.

The system consists of the following main entities:

1. Tutorial
2. TutorialRanking

All data for tutorials and rankings is pre-populated in the database using an SQL file.

There is an existing API endpoint that retrieves all tutorials or tutorials filtered by title. However, this endpoint is not optimized for performance, especially when dealing with a large number of tutorials and rankings.

Question 3.1: Optimize API Performance

Existing API Endpoint

- **HTTP Method**: GET
- **Endpoint**: `/api/tutorials`
- **Query Parameters**:
  - `title` (optional): String to filter tutorials by title

Task

Your task is to optimize the existing `/api/tutorials` endpoint to improve its performance. The optimized endpoint should:

1. Retrieve tutorials with their average ranking score.
2. Handle large datasets efficiently.
3. Minimize database queries and optimize data fetching.

Evaluation Criteria

Your solution will be evaluated based on the following criteria:

1. Correctness of the implementation
2. Performance improvements achieved
3. Code quality and adherence to best practices
4. Proper use of Spring Boot and JPA features

Good luck with your optimization task!

Question 3.2: Implement Caching

Requirements

1. Implement caching for the tutorial management system with the following specifications:
    - Automatically update cached entity objects when corresponding database entities are modified
    - Create an additional API endpoint for updating tutorials to test the cache invalidation
    - Ensure cache consistency with the database

Evaluation Criteria

Your caching implementation will be evaluated based on:

1. Proper implementation of Spring Cache
2. Correct cache invalidation strategy
3. Cache consistency maintenance
4. Performance improvements with caching
5. Proper handling of cache updates when entities are modified

Additional API Endpoint

- **HTTP Method**: PUT
- **Endpoint**: `/api/tutorials/{id}`
- **Request Body**: Tutorial update data
- **Purpose**: Update tutorial information and test cache invalidation